AI WEATHERWISE

Intelligent Weather Forecasts and AI-Powered Weather Insights

Phase 1: Brainstorming & Ideation

Project Documentation

BSc Computer Science Final Year Project

REST API Backend Application with AI-Powered Weather Intelligence

Technology Stack

Node.js | Express.js | MongoDB Atlas | Mongoose | JWT | bcrypt | Google Gemini AI

Document Date: September 28, 2026

Status: Initial Brainstorming Phase

Table of Contents

1. Problem Statement

1.1 Difficulty in Understanding Raw Weather Information

1.2 Difficulty in Managing Multiple Favorite Locations

1.3 Lack of Personalized Weather Recommendations

1.4 Need for Secure Storage and Synchronization of User Preferences

1.5 Need for a Reliable, Secure and Intelligent Weather Intelligence Service

2. Idea Generation

2.1 How the Project Idea Was Developed

2.2 Idea Integration Table

2.3 Integration Summary

3. Final Project Idea

3.1 Project Overview

3.2 Main Objective

3.3 Target Users

3.4 Core Features

4. Expected Benefits

5. AI WeatherWise – Idea Concept Diagram

5.1 Diagram Explanation

6. Phase 1 Conclusion

1. Problem Statement

Weather information is used every day to decide what to wear, where to travel and which outdoor activities are suitable. Although weather data is widely available, users who move between cities or who plan activities around local conditions face a number of practical difficulties in turning that data into decisions. The following subsections describe the specific problems that the AI WeatherWise project is designed to address, based on the scenario of a travel enthusiast who frequently visits different cities and needs a streamlined way to track local weather and receive intelligent activity suggestions.

1.1 Difficulty in Understanding Raw Weather Information

Weather services normally present conditions as numerical metrics such as temperature, humidity and wind speed, together with a short condition label. For many users these values are difficult to interpret in a practical sense. A reading of 34 °C with high humidity, for example, does not by itself tell a person how the day will feel or how to prepare for it. Interpreting a combination of several metrics requires experience and effort, and the user must repeat this interpretation for every city and every time the conditions change.

Basic weather lookups stop at presenting the data. They do not translate it into a readable, natural-language description, so the burden of comprehension remains entirely with the user. This comprehension gap is the first problem that AI WeatherWise is designed to reduce.

1.2 Difficulty in Managing Multiple Favorite Locations

Users who travel or who follow conditions in several cities need to check weather for the same set of places repeatedly. Without a structured system, they must retype city names, search again each time or keep informal lists outside the weather tool. This approach is inefficient and easily leads to inconsistent or duplicated entries.

A dedicated favorite-location facility is required in which each user can save, view, update and remove cities in an organized way, and in which the same city and country cannot be stored twice for one user. Simple weather lookup tools that treat every request as independent do not provide this kind of persistent, user-specific organization.

1.3 Lack of Personalized Weather Recommendations

Knowing the weather is only the first step; users usually want to know what to do about it. Typical questions include what clothing is appropriate, whether hydration is a concern, and whether an outdoor activity is sensible under the current conditions. Conventional weather applications and basic weather APIs generally return data and do not offer condition-aware guidance of this kind.

There is therefore a need for an intelligent component that can take the current conditions of a city and produce personalized, easy-to-follow recommendations for activities and clothing. Generative AI offers a practical way to provide this guidance without the need to hand-code a rule for every combination of weather conditions.

1.4 Need for Secure Storage and Synchronization of User Preferences

Once a user has built a list of favorite locations, that list becomes a personal preference that should be available consistently and protected from other users. Storing preferences only on a single device means they cannot be synchronized across devices, while storing them in a shared, unprotected space allows other people to view or alter them.

A secure, centralized system is needed in which user accounts are protected by hashed passwords, access is controlled through token-based authentication, and every saved location is linked to the user who owns it. Such a system allows preferences to be stored once in a database and accessed from any client that presents a valid token.

1.5 Need for a Reliable, Secure and Intelligent Weather Intelligence Service

The problems above are interconnected. Users need weather data, an understandable interpretation of that data, persistent management of the locations they care about, and protection of their personal information. Providing these capabilities separately leaves gaps: data without interpretation is hard to use, and interpretation without secure user management cannot be personalized.

A further practical concern is dependence on external services. Live weather data and AI-generated text both come from third-party providers, and a backend that stops responding whenever an external weather key is missing or a provider is unavailable is difficult to develop, demonstrate and test. A comprehensive solution should therefore combine secure data management, real-time weather retrieval, AI-generated insight and resilient handling of external data, exposed through a clean and well-structured REST API.

2. Idea Generation

2.1 How the Project Idea Was Developed

The AI WeatherWise idea emerged from a systematic analysis of the problems described in Section 1 and of the technologies that could address each of them. The ideation process combined the following key concepts into one project.

Real-Time Weather Information

The foundation of the project is the retrieval of current weather metrics for any requested city, namely temperature, humidity, wind speed and condition. This provides the factual data on which every other feature depends.

REST API Architecture

A REST API backend built with Node.js and Express.js was chosen as the delivery model. It exposes clearly separated endpoints, returns JSON, and can serve any client application, including API testing tools during development and web or mobile clients in the future.

User Authentication and Secure API Access

To protect personal data, the idea includes user registration and login using JSON Web Tokens (JWT), with passwords hashed using bcrypt before they are stored. Protected routes require a valid Bearer token, while public weather lookups remain available without an account.

Favorite Location Management

Users are able to maintain a personal list of favorite cities. Each saved location is linked to its owning user, and a unique combination of user, city and country prevents duplicate entries.

MongoDB Data Storage

User accounts and favorite locations are stored in MongoDB Atlas and modelled with Mongoose schemas. Schema validation keeps the stored data consistent and maintains the relationship between users and their locations.

Google Gemini AI Integration

Google Gemini AI, accessed through the @google/genai SDK from a dedicated service layer, was selected as the intelligence engine that converts weather information into natural language.

AI-Generated Weather Summaries

Gemini is used to translate raw metrics into an easily understood narrative summary, addressing the comprehension gap identified in Section 1.1.

AI-Powered Weather Recommendations

Given the city, temperature, humidity and condition, Gemini generates condition-aware guidance such as clothing or hydration advice, addressing the need for personalized recommendations identified in Section 1.3.

Resilient Fallback Weather Handling

To keep the API demonstrable and fault-tolerant, the weather endpoint continues to return a structured response, flagged with an isMock indicator, when an external weather API key is not configured.

2.2 Idea Integration Table

Each candidate idea was evaluated for its purpose and benefit and given a decision, as recorded below.

Idea	Purpose	Benefit	Decision
Real-Time Weather Information	Retrieve current temperature, humidity, wind speed and condition for a city	Accurate, current weather data for any requested city	✓ Included
User Authentication	Register and log in users securely	User privacy and account security	✓ Included
Favorite Location Management	Create, view, update and delete saved cities	Organized, user-specific list of locations	✓ Included
AI Weather Summary	Convert raw metrics into a natural-language description	Weather easier to understand	✓ Included
AI Weather Recommendations	Generate condition-aware activity and clothing advice	Practical, personalized guidance	✓ Included
MongoDB Data Storage	Persist users and favorite locations using Mongoose schemas	Consistent, validated and durable data	✓ Included
JWT-Protected API Routes	Require a valid Bearer token for personalized features	Data isolation and unauthorized access prevention	✓ Included
Google Gemini AI Integration	Process weather details through a Gemini service	Intelligent, context-aware output	✓ Included
REST API Architecture	Expose functionality through structured Express.js endpoints	Scalable, modular and testable backend	✓ Included
Resilient Fallback Weather Handling	Return a flagged mock response when no weather API key is set	Uninterrupted API availability during development and testing	✓ Included

2.3 Integration Summary

All ten ideas were integrated into a single, cohesive AI WeatherWise system. The relationship between them can be summarised as the following flow: **User → Authentication → Weather Data and Favorite Locations → MongoDB → Gemini AI → AI Weather Insights → API Response.**

3. Final Project Idea

3.1 Project Overview

AI WeatherWise is an intelligent, weather-focused REST API backend. It combines a Node.js and Express.js server with Google Gemini AI in order to convert raw environmental data into natural-language weather summaries and personalized recommendations. The platform is built around three pillars: real-time weather retrieval, secure per-user favorite-location management, and Gemini-driven insight generation.

The backend follows a layered, MVC-based structure in which routes map requests to controllers, controllers delegate to services and models, and models persist validated data in MongoDB. Because AI WeatherWise is a headless service, the routing layer takes the place of a traditional view. The role of each major technology is as follows:

- **Node.js** – the JavaScript runtime that executes the server-side logic.
- **Express.js** – the web framework used for routing, JSON request handling, middleware and CORS. The backend is configured to run on PORT 5001.
- **MongoDB Atlas with Mongoose** – MongoDB Atlas hosts the database, connected through a connection string held in an environment variable, while Mongoose defines and validates the User and Location schemas.
- **JWT (jsonwebtoken)** – provides stateless token-based authentication and Bearer-token authorization for protected routes.
- **bcrypt** – hashes user passwords one way before they are stored.
- **Weather API (OpenWeatherMap)** – the external source of current weather metrics, accessed through the Weather Service.
- **Google Gemini AI** – generates weather summaries and recommendations through the AI Service using the @google/genai SDK.
- **dotenv** – loads configuration such as PORT, MONGO_URI, JWT_SECRET and GEMINI_API_KEY from environment variables rather than from source code.

3.2 Main Objective

To develop a secure, centralized and scalable weather intelligence REST API that provides real-time weather information, makes that information easier to understand through AI-generated summaries and recommendations, and allows authenticated users to manage a personalized list of favorite locations. Specifically, the project aims to:

- provide current weather metrics for any requested city, with a resilient fallback mode;
- make weather information easier to understand by translating raw data into natural language;
- generate AI-powered weather insights and condition-aware recommendations using Google Gemini AI;
- allow users to create and manage favorite locations that are linked to their own accounts;
- provide secure registration, login and protected functionality using JWT and bcrypt; and

- expose clearly separated public and protected endpoints through a modular, scalable REST API.

3.3 Target Users

- Travel enthusiasts who visit different cities and want to track local weather and receive AI-driven activity suggestions.
- Registered users who want to maintain a personal list of favorite locations and obtain personalized weather summaries and recommendations.
- Casual weather application users who only wish to view current conditions through the public weather endpoint without creating an account.
- Administrators, who are described in the project documentation as responsible for user account management and for monitoring API health and database integrity.
- Developers of future client applications, since the REST API is designed to serve as an extensible foundation for them.

3.4 Core Features

User Authentication

Users can register with a name, email and password and log in with their email and password. Registration validates the required fields and email format, and the password is hashed with bcrypt before it is stored in MongoDB. On a successful login the system issues a signed JWT. Authentication is needed so that personal data such as favorite locations can be tied to, and protected for, one user; it forms the security entry point of the whole system.

Real-Time Weather Information

The weather feature retrieves current metrics (temperature, humidity, wind speed and condition) for a city supplied in the request. It is needed because all summaries and recommendations depend on accurate current data. It is provided through the Weather Controller and Weather Service and is accessible publicly, so users can check conditions without creating an account.

Resilient Fallback Weather Handling

When an external weather API key is not configured, the weather endpoint is designed to keep returning a structured response that is flagged with an isMock indicator instead of failing. This is needed to keep the backend demonstrable and fault-tolerant during development and testing, and it ensures that the remaining features can still be exercised.

Favorite Location Management

Authenticated users are able to create, view, update and delete their favorite locations, each defined by a city and a country. Every location stores a reference to its owner, and a compound unique index on user, city and country prevents duplicates. This feature answers the need for organized, user-specific location management and is delivered through the Location Controller and Location model.

AI-Generated Weather Summary

The AI service is designed to convert weather data (city, temperature, humidity and condition) into a natural-language weather summary. This is needed because raw metrics are difficult to interpret. It follows the same request path as recommendations, through the AI Controller and Gemini Service.

AI Weather Recommendations

Given the city, temperature, humidity and condition, Google Gemini AI generates a personalized, condition-aware recommendation, for example clothing or hydration advice. This feature turns weather knowledge into practical action and is the main intelligent output of the platform. It is exposed through the AI routes.

MongoDB Data Storage

User accounts and favorite locations are persisted in MongoDB Atlas using Mongoose schemas with required fields, unique email addresses and timestamps. This storage is needed for durable, validated data and supports the one-to-many relationship in which a user owns multiple locations.

JWT-Protected API Routes

Authentication middleware verifies the Bearer token in the request before private routes such as favorite location management can be used. Public routes, such as the weather lookup, remain open. This separation protects user-specific data while keeping casual access simple.

Google Gemini AI Integration

A dedicated service layer communicates with Google Gemini AI using the @google/genai SDK, with the API key kept in an environment variable. It is the engine behind both summaries and recommendations and keeps AI logic separate from the controllers.

Standard JSON API Responses

The API returns structured JSON with appropriate HTTP status codes and is supported by centralized error handling and request sanitization. Consistent responses simplify testing with API tools such as Thunder Client and make integration with future client applications easier.

Implementation Status Note

Because this document records the Phase 1 concept, the distinction between what is described and what has been evidenced is stated here for accuracy.

Feature	Status According to Project Source
Registration and login	Implemented; successful responses were observed in API testing (201 Created and 200 OK)
Current weather retrieval	Implemented; tested through the public weather endpoint (200 OK), including the isMock indicator for fallback data
AI weather recommendation	Implemented; tested through the AI recommendation endpoint (200 OK)
AI weather summary	Described as part of the same AI service; a separately tested summary endpoint is not shown in the source

4. Expected Benefits

AI WeatherWise is expected to deliver the following practical benefits. Each benefit is directly connected to a feature described in Section 3.4.

Easier Understanding of Weather Information

AI-generated summaries translate temperature, humidity, wind speed and condition into a readable narrative. Users no longer need to interpret raw metrics for each city, which reduces effort and misunderstanding.

Intelligent AI-Generated Weather Insights

By using Google Gemini AI, the platform provides context-aware interpretation of weather data rather than numbers alone, giving users a clearer understanding of the conditions.

Meaningful Weather Recommendations

Condition-aware activity and clothing suggestions, such as hydration or light-clothing advice, help users decide how to prepare for the weather in a particular city.

Organized Favorite Locations

Favorite location management gives each user a single, organized list of the cities that matter to them, with duplicate entries prevented, so that repeated lookups become simpler.

Secure User-Specific Functionality

bcrypt password hashing, JWT authentication and user-scoped location records mean that personalized data is available only to its owner and that unauthorized access is prevented.

Centralized Weather API Access

Weather retrieval, favorite locations and AI insights are provided through one backend with clearly defined routes, so clients need not integrate with several services individually.

Reliable Operation During Development and Testing

The fallback weather mode allows the API to keep returning structured responses when an external weather key is absent, supporting demonstration and testing.

Scalable REST API Architecture

The layered MVC structure, with separate routes, controllers, services and models, and the use of MongoDB make the system easy to maintain and extend, and allow future web or mobile clients to be added without changing the backend design.

5. AI WeatherWise – Idea Concept Diagram

The diagram below presents the AI WeatherWise system flow from the user's request to the final API response.

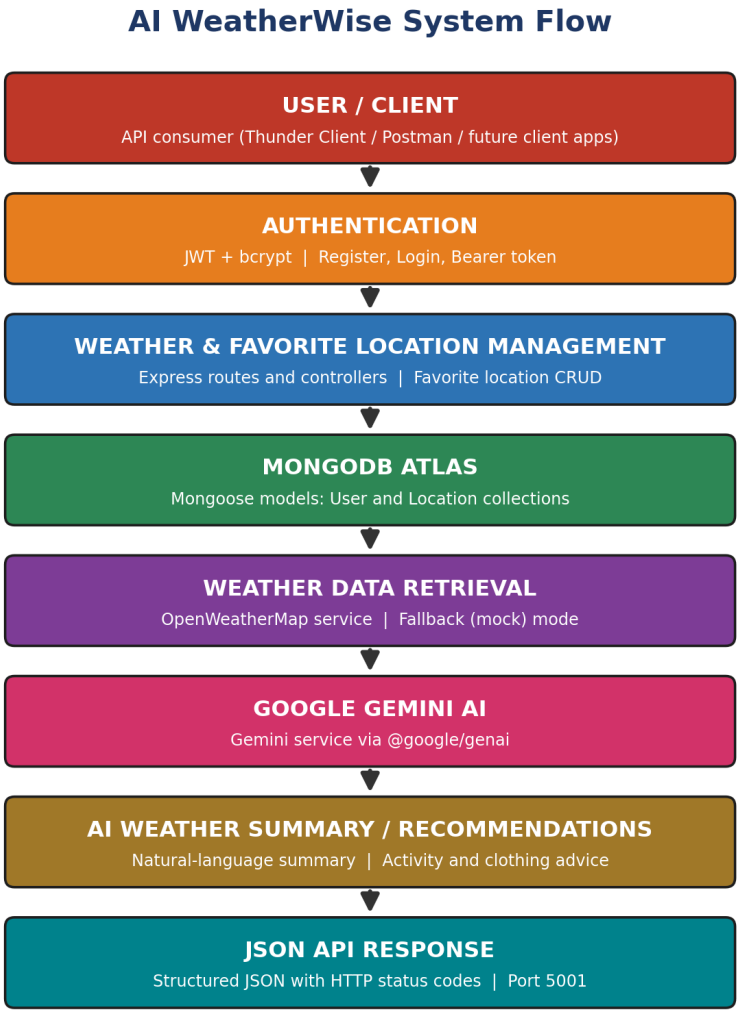


Figure 1: AI WeatherWise System Flow

5.1 Diagram Explanation

The AI WeatherWise Concept Diagram illustrates the complete system flow from user interaction to intelligent output. The public weather lookup does not require authentication; the flow shown is the complete authenticated journey.

User / Client (Red)

The user, or any client application, initiates interaction by sending HTTP requests to the API. During development, API testing tools such as Thunder Client are used as the client. Future web or mobile applications can act in the same role.

Authentication (Orange)

Users register and log in here. Passwords are hashed with bcrypt and a signed JWT is issued on login. The token is then sent as a Bearer token to gain access to protected routes.

Weather & Favorite Location Management (Blue)

Authenticated requests are handled by the Express routes and controllers. This component covers the weather endpoint and the favorite-location create, read, update and delete operations, using authentication middleware to separate public routes from protected routes.

MongoDB Atlas (Green)

MongoDB Atlas stores the User and Location collections modelled by Mongoose. Each location holds a reference to its owning user, which keeps favorite locations scoped to the authenticated user.

Weather Data Retrieval (Purple)

The Weather Service obtains current temperature, humidity, wind speed and condition for a requested city from the OpenWeatherMap API. If no weather key is configured, it returns a flagged mock response so that the API stays available.

Google Gemini AI (Pink)

The Gemini Service passes the weather details to Google Gemini AI through the @google/genai SDK. The AI engine interprets the data and generates natural-language content.

AI Weather Summary / Recommendations (Gold)

The output of the AI stage consists of a readable weather summary and personalized, condition-aware recommendations, for example clothing or hydration advice.

JSON API Response (Cyan)

The result is returned to the client as a structured JSON response with an appropriate HTTP status code. Centralized error handling keeps failures consistent, completing the weather intelligence cycle. The backend listens on PORT 5001.

6. Phase 1 Conclusion

Phase 1 – Brainstorming & Ideation has identified and articulated a clear need for an intelligent, secure and organized approach to weather information. The problem analysis revealed five interconnected challenges:

- Difficulty in understanding raw weather information;
- Difficulty in managing multiple favorite locations;
- Lack of personalized, condition-aware weather recommendations;
- Need for secure storage and synchronization of user preferences; and
- Need for a reliable, secure and intelligent weather service.

Through systematic ideation, these problems were synthesised into a single solution: AI WeatherWise. Ten ideas were evaluated and integrated into one system:

- Real-time weather information for any requested city
- Secure user authentication using JWT and bcrypt
- Favorite location management linked to each user
- AI-generated weather summaries
- AI-powered weather recommendations
- MongoDB Atlas data storage using Mongoose
- JWT-protected API routes with Bearer token authorization
- Google Gemini AI integration through a dedicated service
- A modular REST API architecture
- Resilient fallback weather handling

The resulting concept is an MVC-based REST API backend built with Node.js and Express.js, running on PORT 5001, in which controllers, routes, models, middleware and services are clearly separated. MongoDB Atlas provides persistent, validated storage for users and their favorite locations; bcrypt and JWT protect accounts and personalized routes; weather data is obtained through a weather service with a resilient fallback; and Google Gemini AI converts that data into summaries and recommendations that users can readily understand. Sensitive configuration is kept in environment variables.

The expected benefits, namely easier understanding of weather, intelligent AI-generated insights, meaningful recommendations, organized locations, secure user-specific functionality, centralized API access and a scalable architecture, each follow directly from a defined feature. The project documentation also confirms that registration, login, current weather retrieval and AI weather recommendations have been exercised through API testing, providing early evidence that the concept is practical.